Universität Rostock

UI-DESIGN FÜR INGENIEURANWENDUNGEN

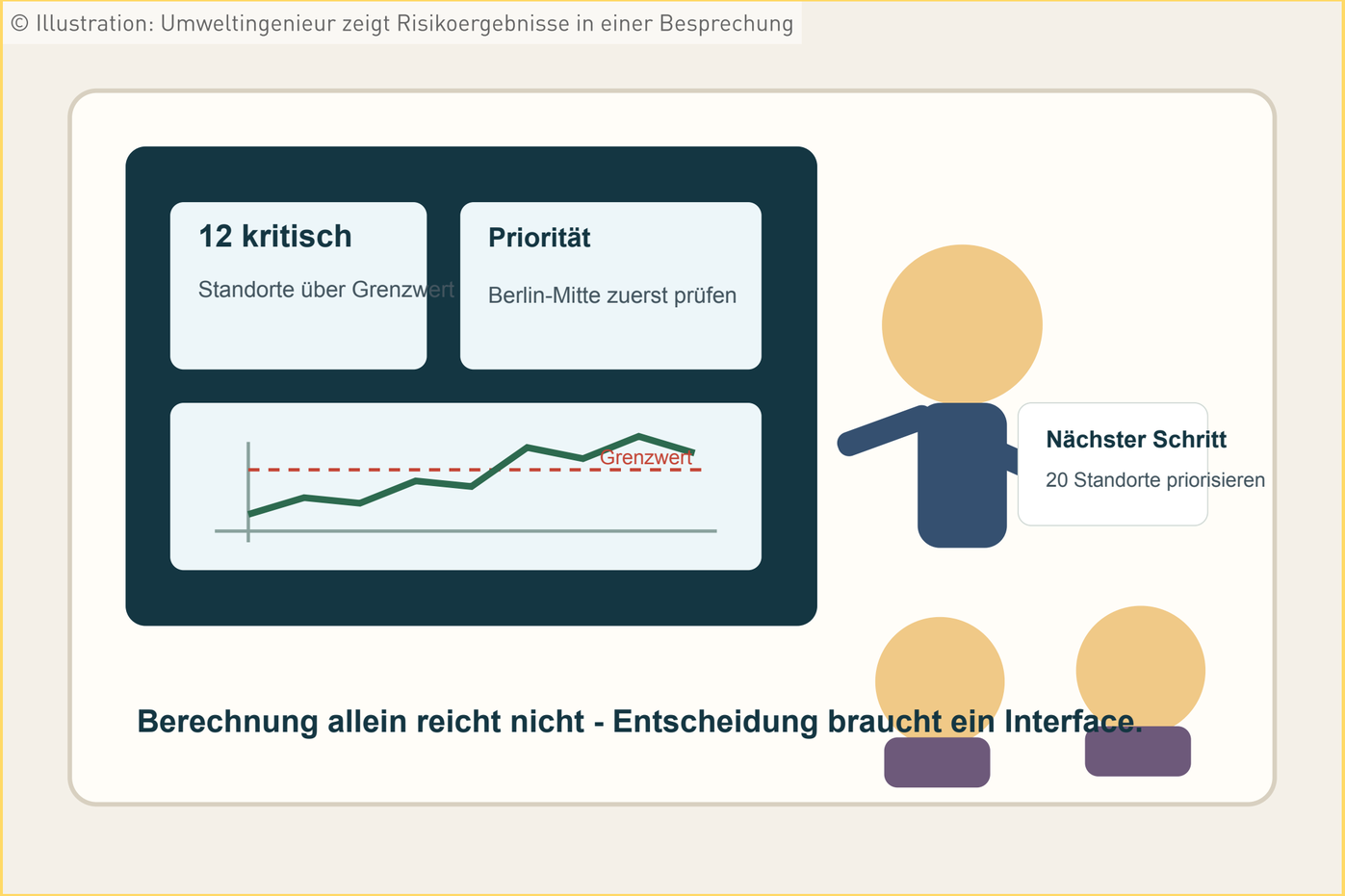
Joern Ploennigs · AI4SC

Midjourney: A futuristic engineering control room filled with sensor dashboards and data monitors

Hörsaalfrage

Du hast die Schadstoffbelastung für 500 Standorte berechnet — wer sieht das Ergebnis, und wie?

© Illustration: Umweltingenieur zeigt Risikoergebnisse in einer Besprechung



500 Standorte, ein CSV — was jetzt?

Das Ergebnis liegt vor:

- 500 Zeilen · 12 Spalten · keine Legende
- Kritische Grenzwertüberschreitungen — irgendwo darin
- Nächste Entscheidung: Welche 20 Standorte priorisieren?

Ohne Interface bleibt die Frage offen.

Eine Zahl ohne Einheit ist keine Messung.

Ein Messwert ohne Kontext ist keine Information.

Eine Information ohne Interface ist keine Entscheidungsgrundlage.

UI-TYPEN

UI-Typen im Vergleich

Format	Interaktiv	Zielgruppe	Typischer Einsatz
Konsole / Terminal	—	Entwickler	Tests, Skripts
Jupyter Notebook	begrenzt	Ingenieure	Analyse, Dokumentation
Streamlit Web-App	**ja**	**Fachpublikum**	**Dashboards, Prototypen**
Gradio Interface	**ja**	**Endnutzer**	**ML- und Modell-Schnittstellen**
PDF-Report	—	Management	Offizielle Berichte

DESIGN-PRINZIPIEN

Design Thinking für Ingenieur-UIs

1. Empathize

Wer nutzt das Interface?

Welche Entscheidungen trifft er?

2. Define

Welche *Mindestinformation* braucht der Nutzer?

(Nicht alle Berechnungs-ergebnisse!)

3. Ideate

Welche Darstellung?

Liniendiagramm, Karte, Tabelle — und warum?

4. Prototype

Streamlit in 30 min

Früh zeigen, früh korrigieren

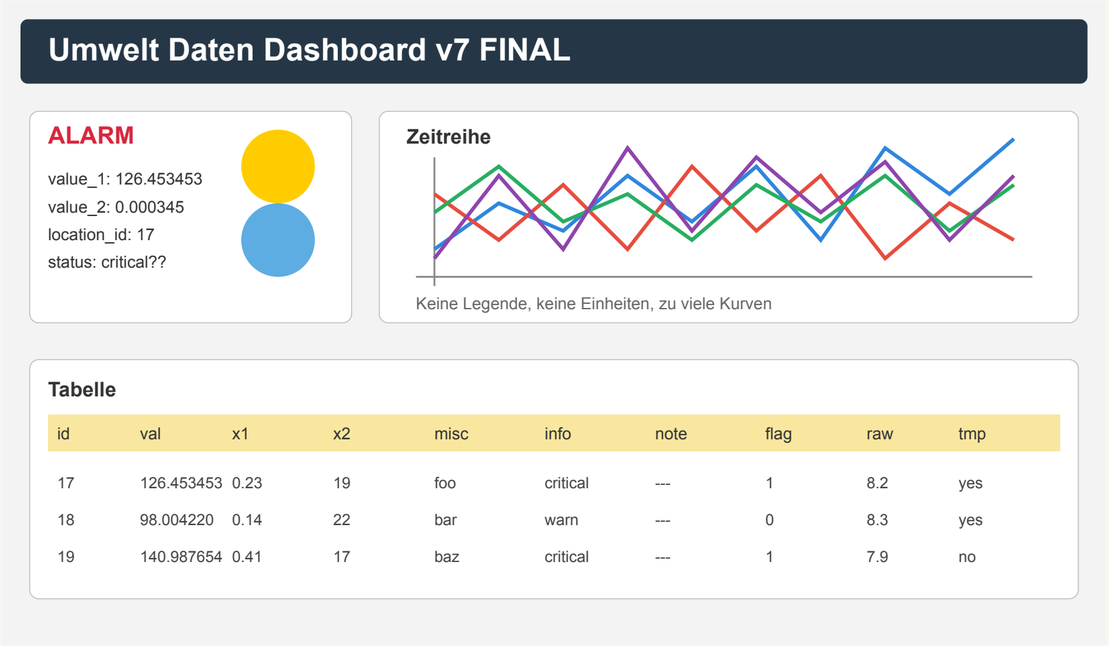
5. Test

Versteht der Nutzer?

Trifft er die *richtige* Entscheidung?

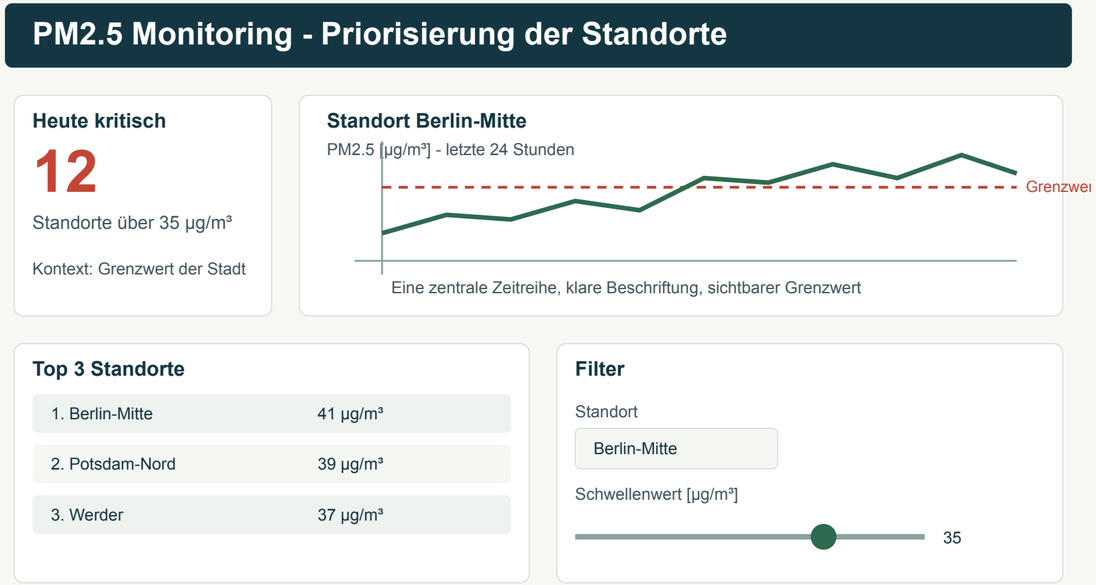
Gute und schlechte Dashboards vergleichen

Schlechtes Dashboard



- kryptische Labels wie `value_1`
- keine Einheiten, keine Legende
- zu viele gleichzeitige Signale

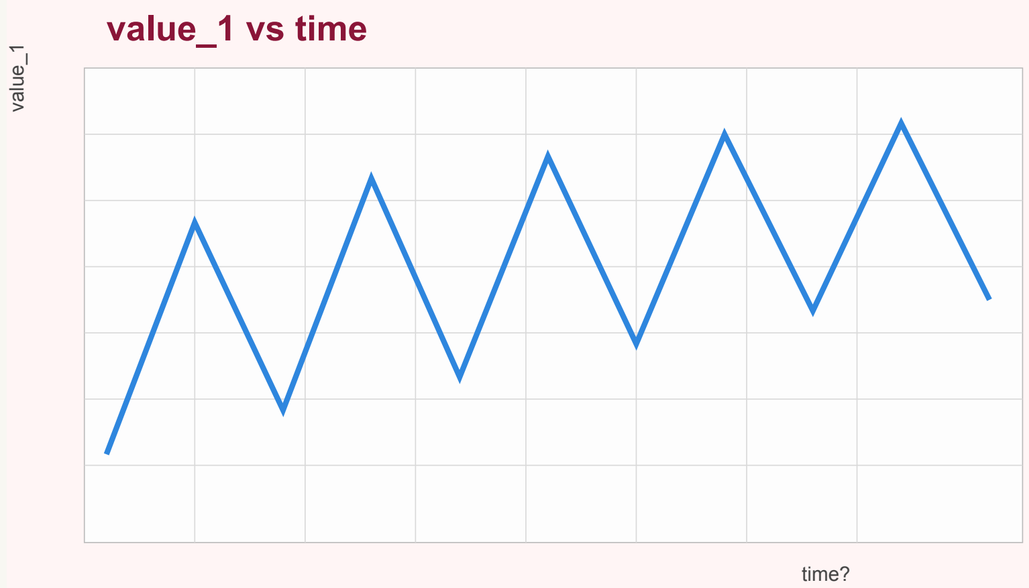
Besseres Dashboard



- klare Priorisierung der Kernfrage
- sichtbarer Grenzwert und Einheiten
- wenige, semantisch geordnete Elemente

Gute und schlechte Diagramme vergleichen

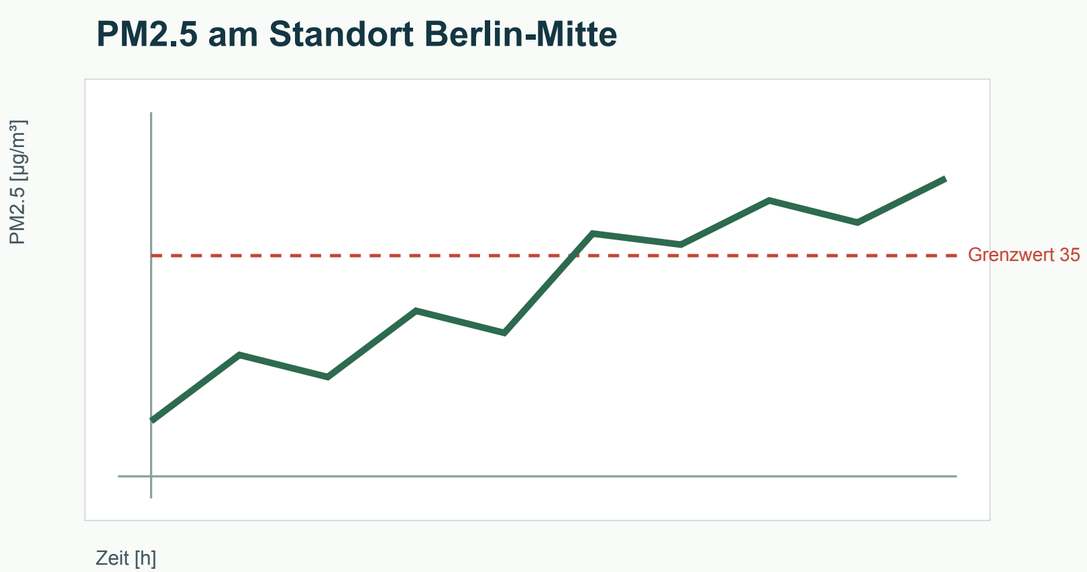
Schlechtes Diagramm



Zu viel Raster, unklare Achsen, keine Einheiten

- Achsen ohne fachliche Bedeutung
- viel Deko, wenig Orientierung
- Referenzwert fehlt

Besseres Diagramm



Weniger Deko, klare Aussage, sichtbarer Referenzwert

- Achsen mit Größe und Einheit
- Grenzwert sichtbar markiert
- eine erkennbare Kernaussage

Grundsätze I: Klarheit und Präzision

Klarheit

- Ein Diagramm = eine Aussage
- Keine 10 Kurven in einem Chart
- Wichtigstes zuerst (*Informationshierarchie*)

Präzision

- Einheiten immer: “PM2.5 [µg/m³]”, nicht “value_1”
- Signifikante Stellen: 2,34 mg/l — nicht 2,3400000
- Achsengrenzen: physikalisch sinnvoll, kein Auto-Scale

Informationshierarchie

- wichtigste Kennzahl nach oben links
- hoher Kontrast für kritische Werte
- Details erst nachrangig sichtbar machen

Affordances

- ein Slider sollte wie ein Slider aussehen
- Buttons eindeutig beschriften
- Interaktion muss erwartbar wirken

Grundsätze II: Wahrnehmung und Reduktion

Gestalt-Prinzipien

- *Proximity:* Zusammengehörige Werte beieinander
- *Similarity:* Gleiche Farbe = gleiche Bedeutung
- *Figure/Ground:* Daten vom Hintergrund abheben
- *Continuity:* Linien als zusammenhängende Verläufe nutzen

Data-Ink-Ratio *(Tufte)*

- Minimaler dekorativer Aufwand
- Maximaler Informationsgehalt
- Jedes Element muss eine Funktion erfüllen

Nielsen-Heuristiken: Anschluss an HCI-Grundlagen

Unser Fokus in dieser Vorlesung	Nielsen-Heuristik
klare Labels, Einheiten, Rückmeldung	1. Sichtbarkeit des Systemstatus
bekannte Begriffe wie “PM2.5 [µg/m³]” statt `value_1`	2. Übereinstimmung mit der realen Welt
vertraute Widgets und klare Bedienung	4. Konsistenz und Standards
wenig Deko, klare Fehlerhinweise, sinnvolle Defaults	8. Ästhetisches und minimalistisches Design
verständliche Fehlermeldungen bei falschen CSVs	9. Fehler erkennen, diagnostizieren, beheben

Unsere Regeln sind also keine Geschmackssache, sondern lassen sich auf etablierte HCI-Heuristiken abbilden.

STREAMLIT DEMO

Agentischer Workflow: Streamlit-Dashboard

Prompt an die KI:

> *“Generiere eine Streamlit-App, die Sensorzeitreihendaten aus einer CSV-Datei lädt, nach Standort filtert und mit einem einstellbaren Schwellenwert als Linie im Plotly-Diagramm anzeigt.”*

Prompt → Generate → Understand → Verify → Iterate

KI liefert ein funktionierendes Grundgerüst in < 1 min.

Der Ingenieur trägt Verantwortung für das Ergebnis.

Nicht: “Läuft es?”

Sondern: “Ist das Ergebnis korrekt?”

- Labels richtig? ✓
- Einheiten angegeben? ✓
- Daten korrekt gemappt? ✓
- Schwellenwert sinnvoll? ✓

Vom Prompt zum ersten Dashboard

1. Daten laden

- CSV auswählen
- Spalten prüfen
- Standort auswählen

2. Messreihe zeigen

- Zeit auf X-Achse
- Messwert auf Y-Achse
- Titel pro Standort

3. Schwellenwert prüfen

- Slider für Grenzwert
- rote Referenzlinie
- Plot gegen Fachfrage lesen

KI-generierter Code: erster Prototyp

```python
# KI-generiert – Streamlit Sensor-Dashboard
import streamlit as st
import pandas as pd
import plotly.express as px

st.title("Sensor-Monitoring")

file = st.file_uploader("CSV laden", type="csv")
if file:
    df = pd.read_csv(file)
    standort = st.selectbox("Standort", df["location"].unique())
    df_filtered = df[df["location"] == standort]
    threshold = st.slider("Schwellenwert", 0, 500, 100)
    fig = px.line(df_filtered, x="timestamp", y="value",
                  title=f"Messwerte – {standort}")
    fig.add_hline(y=threshold, line_color="red")
    st.plotly_chart(fig)
```

Ingenieur prüft (Pattern C)

Was passiert, wenn deine CSV die Spalte `messwert` heißt, nicht `value`?

Welche Einheit hat der Schwellenwert? Steht sie im Label oder Diagramm?

Streamlit verifizieren — was der Ingenieur prüft

Datenmapping

- Spaltennamen im Code ↔ Spalten in der CSV
- Datentypen korrekt? (Datum als `datetime`, Messwert als `float`)
- Fehlende Werte: wie behandelt?

Darstellung

- Y-Achse beschriftet mit Größe und Einheit?
- Achsengrenzen physikalisch sinnvoll?
- Farbskala: Bedeutung klar und konsistent?

Logik

- Schwellenwert in richtiger Einheit?
- Filter-Logik: Ort, Zeit — korrekt umgesetzt?
- Edge cases: leere CSV, ein Standort, kein Grenzwertverstoß?

Nutzerführung

- Fehlermeldungen für Nicht-Entwickler verständlich?
- Ladezeit akzeptabel bei echten Datenmengen?
- Entscheidet ein Nutzer korrekt auf Basis der Anzeige?

Gradio: Interaktive Schnittstellen für Modelle

Gradio eignet sich, wenn ein Simulations- oder ML-Modell direkt interaktiv ausgeführt werden soll — ohne Datei-Upload-Logik.

```python
# KI-generiert – Gradio Abfluss-Rechner
import gradio as gr

def berechne_abfluss(niederschlag, bodentyp):
    faktor = 0.3 if bodentyp == "Ton" else 0.1
    return f"Abfluss: {niederschlag * faktor:.2f} mm"

demo = gr.Interface(
    fn=berechne_abfluss,
    inputs=[gr.Number(label="Niederschlag [mm]"),
            gr.Dropdown(["Sand", "Ton"],
                        label="Bodentyp")],
    outputs=gr.Text(label="Abfluss [mm]")
)
demo.launch()
```

Streamlit — Daten laden, filtern, visualisieren

→ Sensor-Dashboard, Messwertauswertung

Gradio — Parameter eingeben, Modell ausführen

→ Bemessungstools, Simulations-Schnittstellen

Gradio eignet sich besonders für die Weitergabe von Berechnungsmodellen an Fachkollegen ohne Python-Kenntnisse.

VERIFIKATIONS-CHECKLISTE

UI-Verifikations-Checkliste

Beschriftung & Einheiten

- [ ] Achsenbeschriftungen inkl. Einheit (“PM2.5 [µg/m³]”)
- [ ] Achsenbereiche physikalisch sinnvoll
- [ ] Legendeneinträge klar und vollständig
- [ ] Titel beschreibt Inhalt, nicht Technik

Datenmapping

- [ ] Jeder Code-Spaltenname ↔ tatsächliche CSV-Spalte geprüft
- [ ] Daten korrekt gefiltert (Ort, Zeit, Parameter)
- [ ] Aggregation bewusst gewählt (Summe, Mittel — warum?)

Darstellung

- [ ] Farbskala semantisch korrekt (Rot = Alarm, Grün = OK)
- [ ] Kein irreführender Achsenstart (Y beginnt bei 0, wo sinnvoll)
- [ ] Ein Diagramm = eine Aussage (kein Overloading)

Fehler & Robustheit

- [ ] Fehlerzustände für Zielgruppe verständlich
- [ ] Korrekt bei leeren oder fehlerhaften Daten
- [ ] Grenzwert-Logik mit bekannten Testdaten verifiziert

Grundregel

Falsche Daten klar darzustellen ist gefährlicher als richtige Daten unübersichtlich zu lassen. Das Datenmapping ist immer Ingenieurverantwortung.

Ausblick: Block 6 — Persistente Datenhaltung

Das Dashboard funktioniert. Aber:

- 10 Millionen Messwerte — aus Dateien laden?
- 50 Nutzer gleichzeitig — auf eine CSV?
- Historische Abfragen über Monate — durch alle Ordner?

Dateien skalieren nicht.

→ Block 6: **Datenbanken** als persistente, skalierbare Datenschicht hinter dem Dashboard.

Heute: CSV-Datei → Streamlit

Block 6: Datenbank → Streamlit

Ein Streamlit-Dashboard ist so gut wie seine Datenquelle.

Für Produktivsysteme: Datenbanken statt CSV-Dateien.

QUESTIONS